

React Notes

Chapter 1: Introduction to React

React is an open-source JavaScript library developed by Meta for building user interfaces. It enables developers to create reusable UI components and manage application state efficiently. React follows a component-based architecture where complex interfaces are broken into smaller, manageable pieces.

Traditional web applications frequently manipulate the DOM directly, which can become slow and difficult to maintain as applications grow. React introduces the Virtual DOM, a lightweight representation of the real DOM. When state changes occur, React compares the updated Virtual DOM with the previous version and updates only the necessary elements in the browser.

Advantages of React

- Component-based architecture
- Reusable code
- Improved performance through Virtual DOM
- Strong ecosystem and community support
- Easy integration with APIs and backend services

Real-World Use Cases

React is widely used for:

- E-commerce applications
- Social media platforms
- Learning management systems
- Dashboards and analytics platforms
- SaaS applications

Chapter 2: JSX

JSX stands for JavaScript XML. It allows developers to write HTML-like syntax directly within JavaScript code.

Example:

```
function Welcome() {  
  return <h1>Welcome to React</h1>;  
}
```

JSX improves readability and simplifies UI creation. Under the hood, JSX is converted into JavaScript function calls.

Rules of JSX

- Return a single parent element
- Close all tags properly
- Use camelCase for attributes
- Embed JavaScript expressions using curly braces

Example:

```
const name = "Ebin";

function App() {
  return <h1>Hello {name}</h1>;
}
```

Chapter 3: Components

Components are independent building blocks of a React application.

Functional Components

Modern React primarily uses functional components.

```
function Profile() {
  return <h2>User Profile</h2>;
}
```

Props

Props are read-only values passed from parent components to child components.

```
function User(props) {
  return <h1>{props.name}</h1>;
}
```

Example usage:

```
<User name="Ebin" />
```

Component Composition

Applications are built by combining multiple small components into larger structures.

Benefits:

- Reusability
- Maintainability
- Separation of concerns

Chapter 4: State Management

State represents dynamic data that can change over time.

useState Hook

```
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <button onClick={() => setCount(count + 1)}>
      {count}
    </button>
  );
}
```

State Principles

- Never mutate state directly
- Use setter functions
- Keep state minimal
- Derive values when possible

Common Mistakes

Incorrect:

```
count++;
```

Correct:

```
setCount(count + 1);
```

Chapter 5: useEffect and Lifecycle

The useEffect hook handles side effects such as API calls, subscriptions, and timers.

Example:

```
useEffect(() => {  
  fetchData();  
}, []);
```

Dependency Array

```
useEffect(() => {  
  console.log(user);  
}, [user]);
```

The effect runs whenever the dependency changes.

Cleanup Functions

```
useEffect(() => {  
  const timer = setInterval(fetchData, 1000);  
  
  return () => clearInterval(timer);  
}, []);
```

Cleanup prevents memory leaks.

Chapter 6: Forms and Event Handling

React uses synthetic events for handling user interactions.

Example:

```
<button onClick={handleClick}>
  Submit
</button>
```

Controlled Components

```
const [email, setEmail] = useState("");

<input
  value={email}
  onChange={(e) => setEmail(e.target.value)}
/>
```

Benefits:

- Easier validation
- Better state control
- Predictable behavior

Chapter 7: React Router

React Router enables client-side navigation.

Installation:

```
npm install react-router-dom
```

Example:

```
<Route path="/dashboard" element={<Dashboard />} />
```

Dynamic Routes

```
<Route path="/user/:id" element={<User />} />
```

Navigation

```
<Link to="/dashboard">
  Dashboard
</Link>
```

Chapter 8: API Integration

Most applications communicate with backend services.

Example:

```
async function fetchUsers() {
  const response = await fetch("/api/users");
  const data = await response.json();
}
```

Loading States

```
if (loading) return <p>Loading...</p>;
```

Error Handling

```
try {
  await fetchUsers();
} catch (error) {
  console.error(error);
}
```

Chapter 9: Performance Optimization

React.memo

Prevents unnecessary re-renders.

```
export default React.memo(Component);
```

useMemo

Caches expensive computations.

```
const value = useMemo(() => {  
  return calculate();  
}, [data]);
```

useCallback

Caches function references.

```
const handler = useCallback(() => {  
  doSomething();  
}, []);
```

Lazy Loading

```
const Dashboard = React.lazy(() =>  
  import("./Dashboard")  
);
```

Chapter 10: Project Structure and Best Practices

Recommended structure:

```
src/  
├─ components/  
├─ pages/  
├─ hooks/  
├─ services/  
├─ utils/  
├─ assets/  
└─ App.jsx
```

Best practices:

- Keep components small
- Avoid duplicated logic
- Use custom hooks

- Validate props
 - Handle errors properly
 - Follow consistent naming conventions
 - Write maintainable code
-

Summary

React is a modern JavaScript library used to build scalable user interfaces through reusable components. Core concepts include JSX, components, props, state, hooks, routing, API integration, and performance optimization. Mastering these concepts enables developers to build professional web applications and serves as a foundation for advanced frameworks such as Next.js.